

삼성청년 SW·AI아카데미

B형 특강

<알림>

본 강의는 삼성청년SW·AI아카데미의 콘텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로 복사, 촬영,
녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를
이용한 보관, 제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

Pro 연습 문제. 외계생명체

외계생명체 - 문제 이해

2차원 평면 상에 한 구역에 존재하는 차량에 대한 데이터가 있다.
모든 차량은 각각 다른 위치에 있다.

각 차량은 모두 다른 차 종일수도 있다.

차량에 대한 정보는 1~10이하의 ID로 표현한다.

[Fig. 1]은 구역에 존재하는 차량에 대한 데이터를 점으로 나타낸다.

서로 다른 색의 점은 서로 다른 종류의 차량을 표현한다.

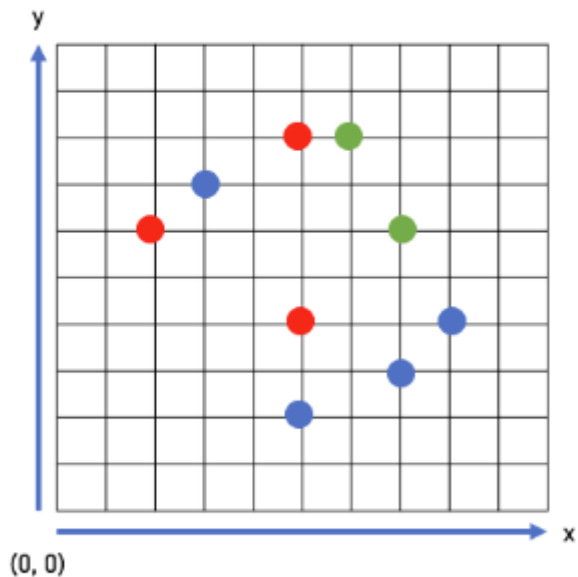


Fig. 1

점 A(aX, bX)와 점 B(bX, BY)의 거리는 $|aX - bX| + |aY - bY|$ 로 정의한다.

예를 들어 점 (1, 2)와 점(5, 5)의 거리는 $|1 - 5| + |2 - 5| = 7$ 이다.

외계생명체 - 문제 이해

외계 생명체는 우주에서부터 (x, y) 위치에 도착한다.

그리고 외계 생명체는 정체를 숨기기 위해 주변 가까이 있는 물체의 데이터를 토대로 변신한다.

마침 주변에는 데이터를 확보할 수 있는 여러 차량에 대한 정보가 존재한다.

변신을 하기 위해서는 가까운 K개의 차량에 대한 데이터를 샘플로 획득해야만 한다.

만약 거리가 같을 때에는, x축이 더 작은 차량을, 그리고 x축의 위치가 같을 때에는 y축의 위치가 더 작은 차량을 우선적으로 샘플로 선정한다.

그리고 K개의 샘플의 데이터를 토대로 가장 많이 포화된 종류의 차량으로 변신한다.

만약 K개의 샘플 데이터에서 가장 많이 포화된 차량이 여러 개 있을 경우, 가장 작은 ID값을 가지는 차량으로 변신한다.

[Fig. 2]는 외계 생명체가(5, 6) 위치에 도착했을 때 주변의 차량에 대한 데이터를 나타낸 것이다.

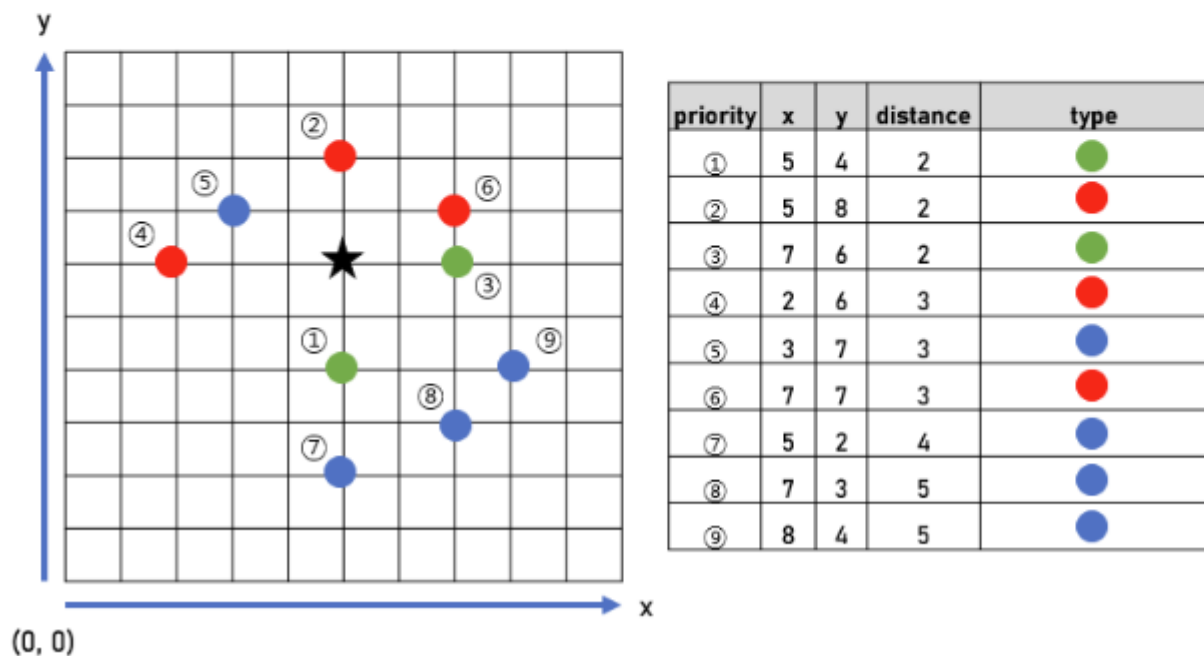
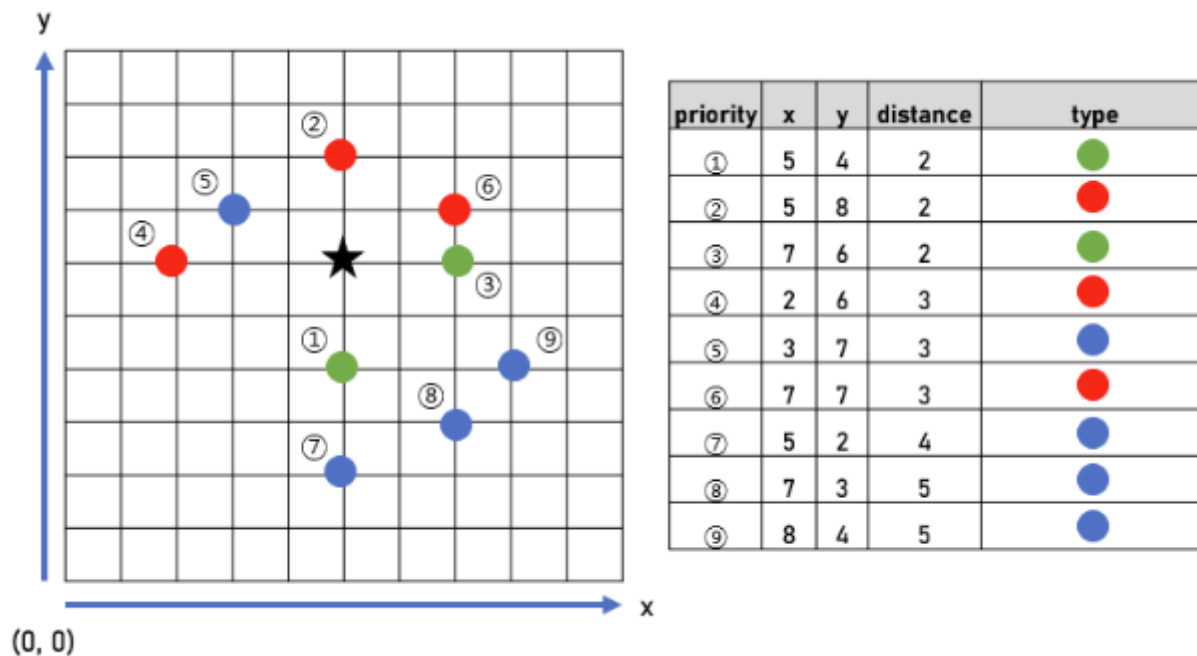


Fig. 2

외계생명체 - 문제 이해



[Fig. 2]의 정보를 토대로,

1개의 샘플이 필요할 경우, 초록색 1개가 있으므로, 초록색에 해당하는 차량으로 변신한다.

3개의 샘플이 필요할 경우, 초록색 2개와 빨간색 1개가 있으므로, 초록색에 해당하는 차량으로 변신한다.

5개의 샘플이 필요할 경우, 초록색 2개와 빨간색 2개가 있으므로, 둘 중 더 작은 ID를 가지는 값으로 변신한다. 예를 들어 초록색이 1, 빨간색이 2라고 가정하면, 초록색에 해당하는 차량으로 변신한다.

7개의 샘플이 필요할 경우, 빨간색 3개가 있으므로 빨간색에 해당하는 차량으로 변신한다.

9개의 샘플이 필요할 경우, 파란색 4개가 있으므로 파란색에 해당하는 차량으로 변신한다.

외계 생명체는 L만큼의 가시거리가 존재한다.

이 L 거리를 벗어나는 차량에 대한 정보는 샘플로 활용할 수 없다.

예를 들어 L = 4이고, 7개의 샘플이 필요할 경우, 7개의 차량에 대한 샘플을 획득할 수 있으므로, 빨간색에 해당하는 차량으로 변신할 수 있다.

단, L = 4이고, 9개의 샘플이 필요할 경우, 9개의 샘플을 획득하지 못하므로 변신이 불가능하다.

외계생명체 - API

```
void init(int K, int L)
```

테스트 케이스에 대한 초기화 함수. 각 테스트 케이스의 맨 처음 1회 호출된다.

- K는 외계 생명체가 변신을 위해 필요한 샘플의 개수이다.
- L는 외계생명체의 가시 거리이다.
- 초기에는 차량이 존재하지 않는다.

Parameters

- K : 외계 생명체가 변신을 위해 필요한 샘플의 개수 ($3 \leq K \leq 11$)
- L : 외계 생명체의 가시 거리 ($4 \leq L \leq 100$)

외계생명체 - API

```
void enter(int mID, int mX, int mY, int mC)
```

ID가 mID이고 x축 위치가 mX이고 y축 위치가 mY이고 종류가 mC인 차량이 구역에 들어온다.

- 함수 호출 시 전달되는 mID 값은 이전 호출에서 이미 전달된 값이 아님을 보장한다.
- 함수가 호출될 때 점 (mX, mY)에 자료는 존재하지 않음이 보장된다.

Parameters

- mID : 구역에 들어올 차량 ID ($1 \leq mID \leq 1,000,000,000$)
- mX : 차량의 x축 위치 ($1 \leq mX \leq 4,000$)
- mY : 차량의 y축 위치 ($1 \leq mY \leq 4,000$)
- mC : 차량의 종류 ($1 \leq mC \leq 10$)

```
void leave(int mID)
```

ID가 mID인 차량이 구역을 떠난다.

- mID는 구역에 존재하는 차량임이 보장된다.

Parameters

- mID : 구역을 떠나는 차량의 ID ($1 \leq mID \leq 1,000,000,000$)

외계생명체 - API

```
int transform(int mX, int mY)
```

외계 생명체가 (mX, mY) 위치에 도착했을 때, 이 위치에서 획득한 K개의 샘플을 기반으로 변신하게 될 차량 종류의 정보를 반환한다.

- 만약 변신을 하지 못할 경우 -1을 반환한다.
- K개의 샘플을 활용한 변신 과정과 가시거리에 대해서는 본문 설명을 참고하라.
- 함수가 호출될 때 존재하는 차량의 수는 K개 이상이고 점 (mX, mY)에 차량은 존재하지 않음이 보장된다.

Parameters

- mX : x축 위치 ($1 \leq mX \leq 4,000$)
- mY : y축 위치 ($1 \leq mY \leq 4,000$)

Returns

- 외계 생명체가 변신하게 될 차량의 종류. 변신을 하지 못할 경우 -1.

[제약사항]

1. 각 테스트 케이스 시작 시 init() 함수가 한 번 호출된다.
2. 각 테스트 케이스에서 enter() 함수의 호출 횟수는 20,000 이하이다.
3. 각 테스트 케이스에서 leave() 함수의 호출 횟수는 1,000 이하이다.
4. 각 테스트 케이스에서 transform() 함수의 호출 횟수는 10,000 이하이다.
5. 임의 위치에 있는 크기가 $100 * 100$ 인 정사각형 안에 있는 차량의 개수는 최대 500이다.

외계생명체 - 예시

[예제]

#	Function	return	Fig
1	init(3, 5)		
2	enter(1, 7, 6, 1)		
3	enter(2, 7, 7, 2)		
4	enter(3, 5, 8, 2)		
5	transform(5, 6)	2	Fig. 3
6	enter(4, 5, 4, 1)		
7	transform(5, 6)	1	Fig. 4
8	enter(5, 2, 6, 2)		
9	transform(4, 7)	2	Fig. 5
10	transform(4, 3)	-1	Fig. 6
11	enter(6, 8, 4, 3)		
12	transform(7, 5)	1	Fig. 7
13	leave(1)		
14	transform(9, 5)	1	Fig. 8
15	enter(7, 7, 6, 3)		
16	transform(9, 5)	3	Fig. 9

(순서 1) 외계 생명체는 3개의 샘플이 필요하고, 5의 가시거리를 가진다.

초기에 차량은 존재하지 않는다.

(순서 2) ID가 1이고 (7, 6) 위치에 종류가 1인 차량이 구역에 들어온다.

(순서 3) ID가 2이고 (7, 7) 위치에 종류가 2인 차량이 구역에 들어온다.

(순서 4) ID가 3이고 (5, 8) 위치에 종류가 2인 차량이 구역에 들어온다.

(순서 5) (5, 6) 위치에서 가장 가까운 3개의 샘플의 데이터에 의해 종류가 2인 차량으로 변신한다.

[Fig. 3]은 함수 호출의 결과를 보여준다.

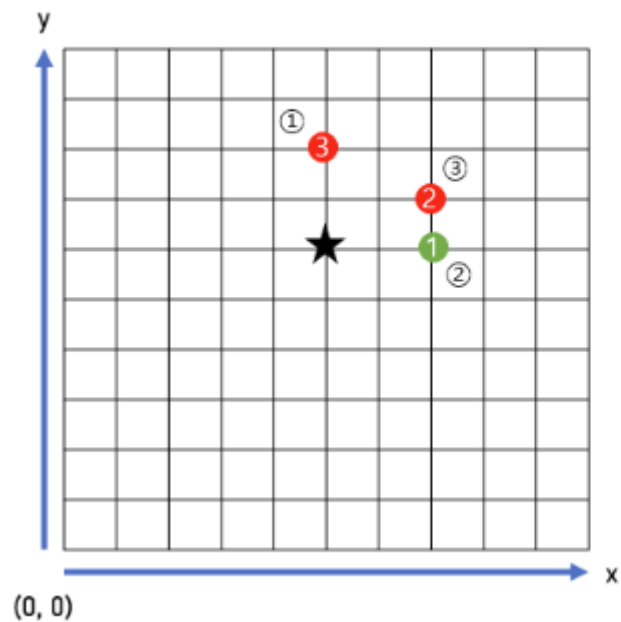


Fig. 3

1 ●
2 ●

priority	x	y	distance	type
①	5	8	2	3
②	7	6	2	1
③	7	7	3	2

외계생명체 - 예시

(순서 6) ID가 4이고 (5, 4) 위치에 종류가 1인 차량이 구역에 들어온다.

(순서 7) (5, 6) 위치에서 가장 가까운 3개의 샘플의 데이터에 의해 종류가 1인 차량으로 변신한다.

[Fig. 4]은 함수 호출의 결과를 보여준다.

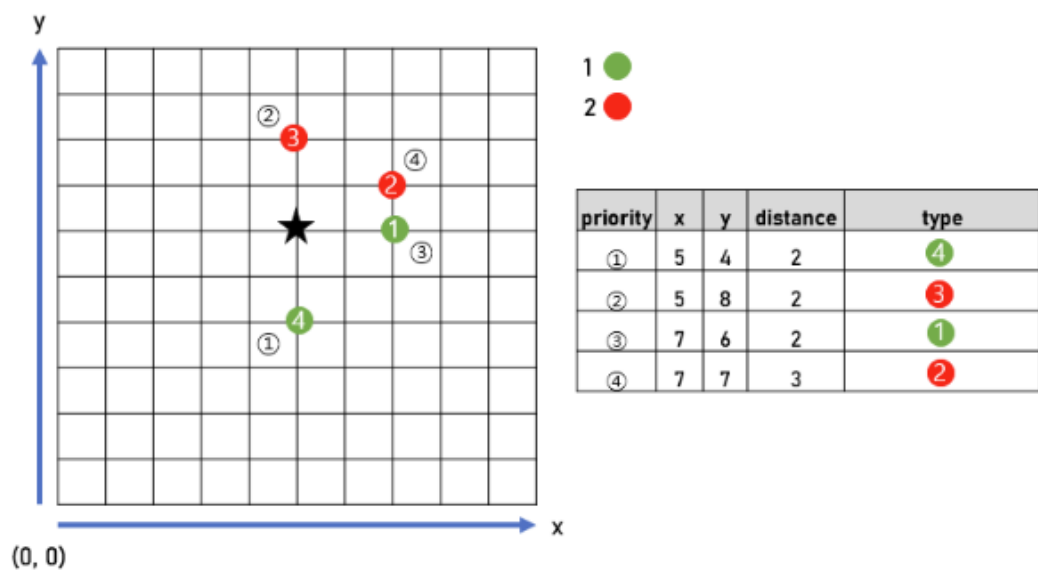


Fig. 4

(순서 8) ID가 5이고 (2, 6) 위치에 종류가 2인 차량이 구역에 들어온다.

(순서 9) (4, 7) 위치에서 가장 가까운 3개의 샘플의 데이터에 의해 종류가 2인 차량으로 변신한다.

[Fig. 5]은 함수 호출의 결과를 보여준다.

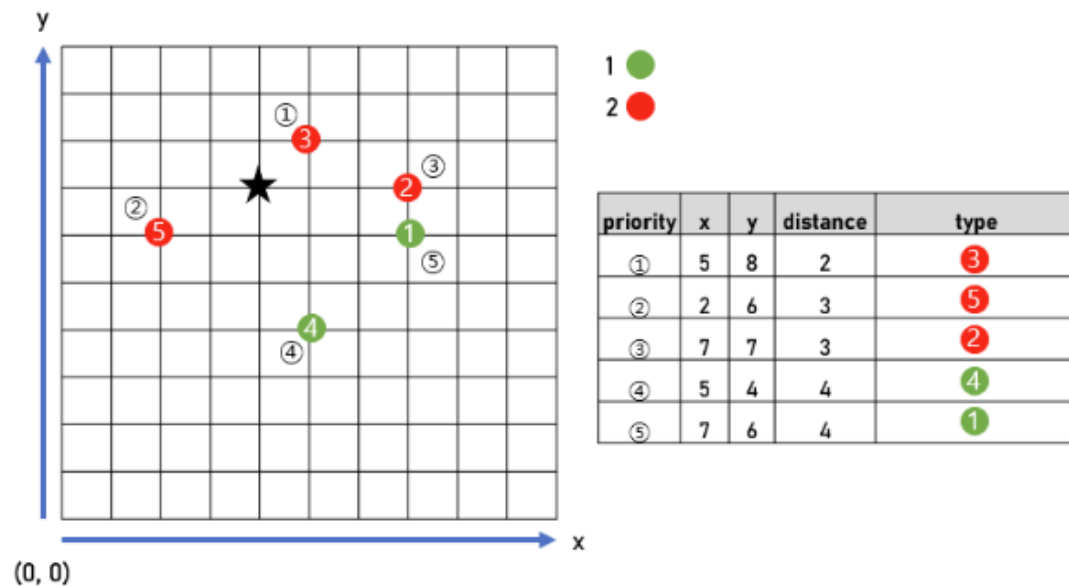


Fig. 5

외계생명체 - 예시

(순서 10) (4, 3) 위치에서는 가시거리에 의해 3개의 샘플을 획득하지 못하여 변신이 불가능하다.

[Fig. 6]은 함수 호출의 결과를 보여준다.

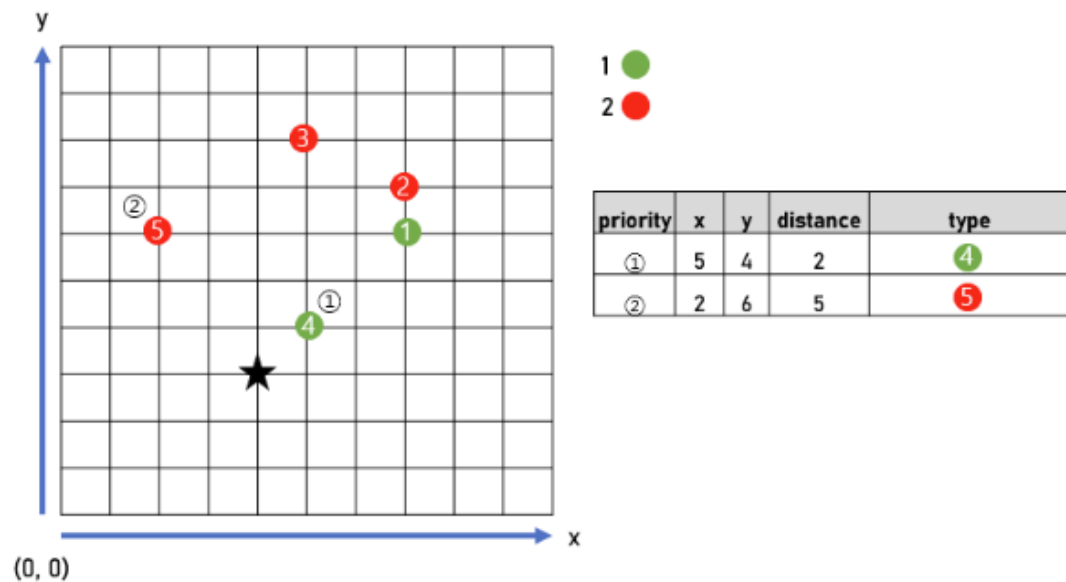


Fig. 6

(순서 11) ID가 6이고 (8, 4) 위치에 종류가 3인 차량이 구역에 들어온다.

(순서 12) (7, 5) 위치에서 가장 가까운 3개의 샘플의 데이터는 1, 2, 3의 종류의 차량이 각각 1개씩 존재한다
그렇기에 우선순위에 따라 종류가 1 차량으로 변신한다.

[Fig. 7]은 함수 호출의 결과를 보여준다.

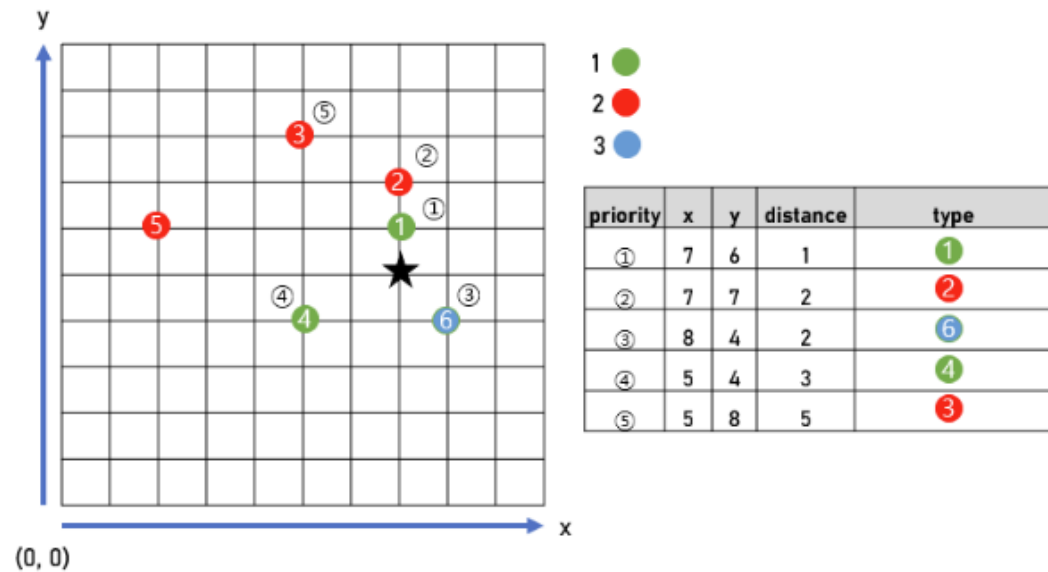


Fig. 7

외계생명체 - 예시

(순서 13) ID가 1인 차량이 구역을 떠난다.

(순서 14) (9, 5) 위치에서 가장 가까운 3개의 샘플의 데이터는 1, 2, 3의 종류의 차량이 각각 1개씩 존재한다. 그렇기에 우선순위에 따라 종류가 1 차량으로 변신한다.

[Fig. 8]은 함수 호출의 결과를 보여준다.

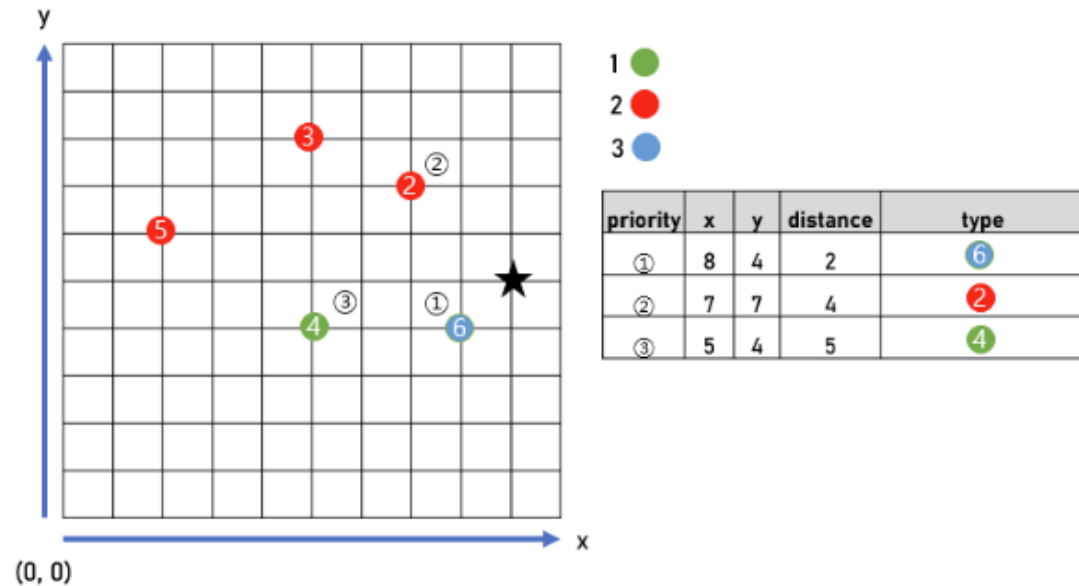


Fig. 8

(순서 15) ID가 7이고 (7, 6) 위치에 종류가 3인 차량이 구역에 들어온다.

(순서 16) (9, 5) 위치에서 가장 가까운 3개의 샘플의 데이터에 의해 종류가 3인 차량으로 변신한다.

[Fig. 9]은 함수 호출의 결과를 보여준다.

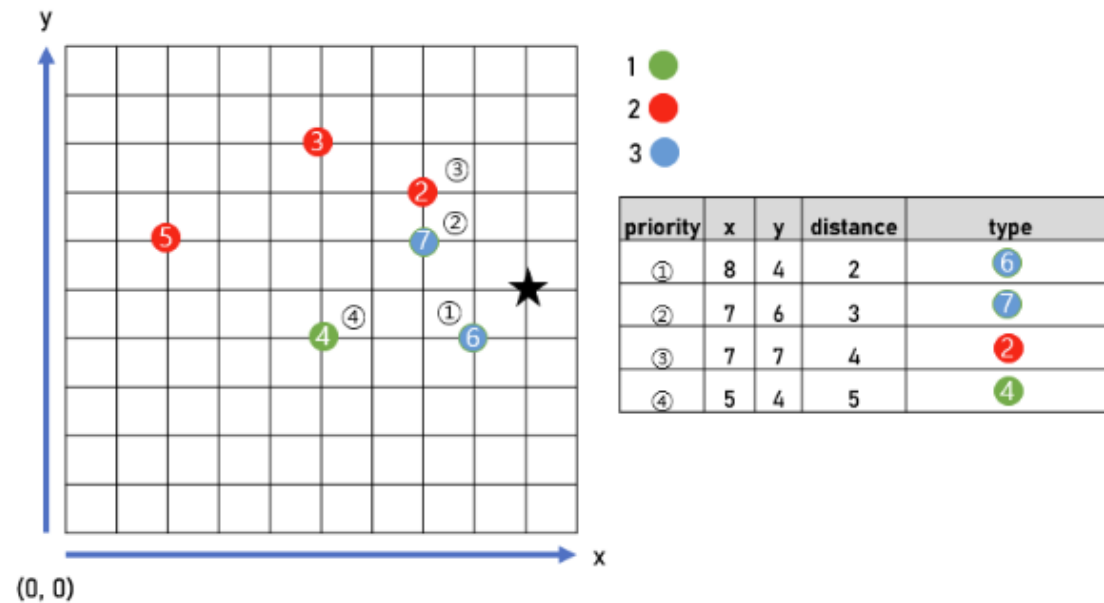


Fig. 9

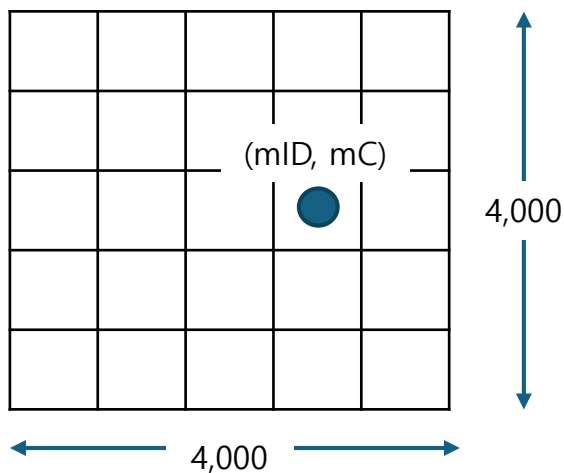
설계하기

삼성청년SW·AI아카데미

enter(), leave() 계산

❖ enter() :

- mID 샘플이 (mY, mX) 에 있고, mC 카테고리를 가진다
- mID 샘플이 어디있는지 관리 필요
- 시간복잡도 : $O(1)$



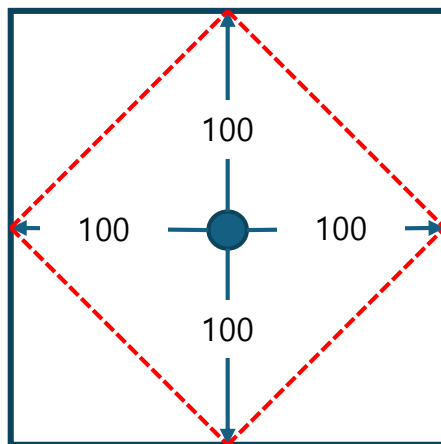
❖ leave():

- mID 샘플이 어딴는지 확인하고, 맵에서 지운다.
- 시간복잡도 : $O(1)$

transform() 계산

❖ transform() :

- (mY, mX) 위치에서 가장 가까운 K개의 샘플을 찾는다.
- 탐색해야 하는 범위는 (mY, mX)로부터 L(=최대100)거리만큼 떨어진 공간이다.



$$= 200 \times 200 / 2 = \text{약 } 20,000\text{칸의 영역}$$

- 카테고리 분류는 그렇다 치고, 일단 간략 계산을 해보면 :
 - 호출횟수 : 10,000
 - 시간복잡도 : $10,000 \times 20,000 = 200,000,000$ (TLE)

- ❖ 20,000칸을 매번 보면 시간 초과가 발생한다.
- ❖ plan #2. 존재하는 샘플들을 모두 비교해보자.
 - 샘플의 개수는 최대 20,000개 (enter()의 호출 횟수)
 - 샘플의 개수가 적을때에는 적게 보겠지만, 20,000개일때에는 20,000개를 보게 된다.
 - 전 설계의 확정적 20,000칸 확인보다는 경우에 따라 적을 수 있겠지만, 최악의 경우 20,000개의 sample이 enter한 후 transform()이 10,000번 호출되면 동일하다.
 - 그래도 여기서는 "경우에 따라" 개선될 수 있음을 확인했다.
- 그렇다면 항상 더 적은 개수의 sample들을 확인할 수 있는 방법이 있을지를 생각해보자.

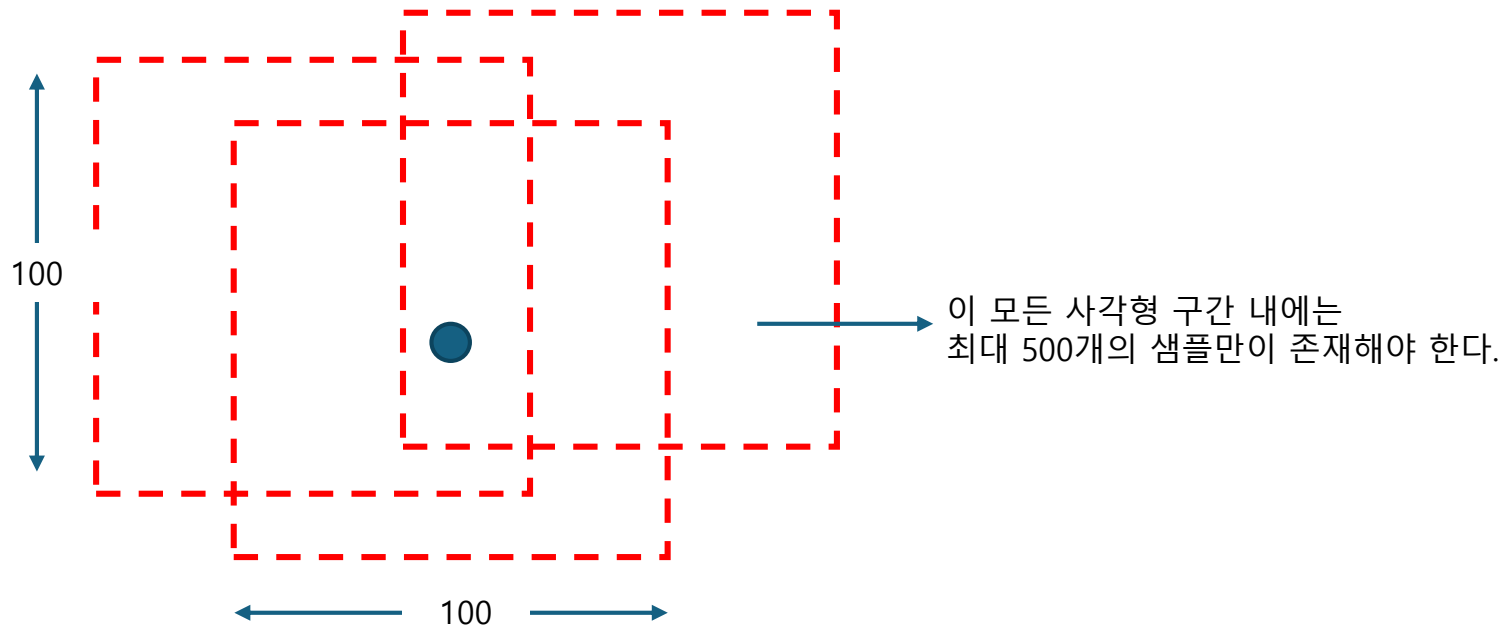
문제는 항상 힌트를 제공한다.

❖ 문제에서 주어지는 힌트를 확인하자.

[제약사항]

1. 각 테스트 케이스 시작 시 `init()` 함수가 한 번 호출된다.
2. 각 테스트 케이스에서 `enter()` 함수의 호출 횟수는 20,000 이하이다.
3. 각 테스트 케이스에서 `leave()` 함수의 호출 횟수는 1,000 이하이다.
4. 각 테스트 케이스에서 `transform()` 함수의 호출 횟수는 10,000 이하이다.
5. 임의 위치에 있는 크기가 $100 * 100$ 인 정사각형 안에 있는 차량의 개수는 최대 500이다.

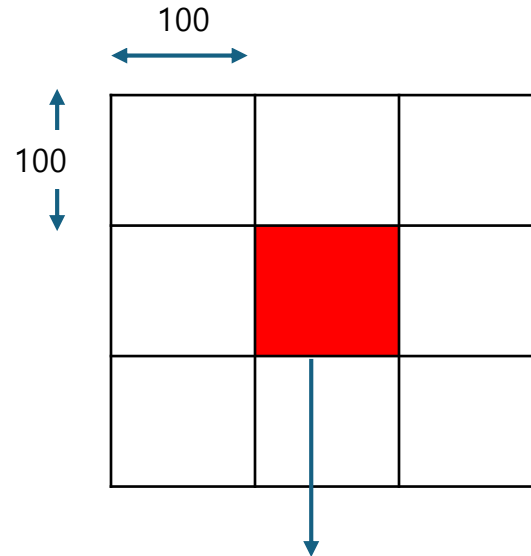
- 어떤 경우라도 100×100 영역 내에는 최대 500개의 샘플/차량만이 존재함이 보장된다



Spatial Partitioning

❖ (mY, mX) 좌표로부터 100 거리 내에 존재하는 샘플들만 살펴보자

- 맵 정보를 100 x 100 공간들로 분할하여, 해당 영역에 살아가는 차량/샘플 정보를 관리한다.
- Spatial Partitioning (공간 분할) 기법이다.



이 영역에 사는 데이터는 $L = 100$ 이라는 가정하에,
주변 공간들에 존재하는 데이터까지 가시거리가 닿을 수 있다.

- 해당 경우, 각 영역에 존재할 수 있는 데이터의 개수는 최대 500이다
- 9개의 공간을 확인하니, 최대 $9 \times 500 = 4,500$ 개의 데이터만 확인하면 된다.

❖ enter()

- (mY, mX)가 존재하는 영역을 확인한다.
- 해당 영역에 샘플의 정보를 투입한다.
- mID는 1~10억이니 id sequencing을 해준다.
- 시간복잡도 : $O(1)$

❖ leave()

- enter()시 투입되었던 정보를 삭제하는데에는 시간이 걸린다 ($O(N)$)
- lazy deletion을 사용하여 시간을 확보한다
- 시간복잡도 : $O(1)$

❖ transform()

- (mY, mX)가 존재하는 영역을 확인한다.
- 해당 영역으로부터 인접한 8개의 영역을 포함한 총 9개의 영역에 존재하는 데이터를 확인한다
- 샘플의 거리가 가시거리 내에 닿을 수 있다면, 가장 가까운 (우선순위) 샘플을 K개 뽑을 수 있도록 Priority Queue에 투입한다.
- 탐색이 끝난 후, Priority Queue에서 순차적으로 K개를 빼내며, 카테고리를 분류한다.

• 시간복잡도 :

- 영역에 존재하는 데이터 확인 : 4,500
- 해당 데이터들이 Priority Queue에 투입되는 시간 : $O(\log 4,500)$
 - 하지만 100 거리내로 존재하는 데이터는 500개임이 보장, 아무리 많아도 500개가 들어갈 것
- K개를 추출하는 시간 $O(K \times \log(4,500)) \rightarrow$ 시간복잡도 계산에서 제외
- 호출 횟수 : 10,000
 - $10,000 \times (4,500 + \log(500)) = \sim 45,000,000$
 - 각 샘플당 pq에 들어가는 것이 아니므로 곱셈이 아니다.
 - Priority Queue에 들어가는 시간복잡도는 $O(\log(1)..\log(2)..\log(3..))$ 과 같이 천천히 늘어난다.